

Entregable 1 – Monitor

Julen García Muñoz

24 de abril de 2026

Índice

1. Módulo Monitor	3
1.1. Descripción General	3
1.2. Constantes	3
1.2.1. Constantes NTC	3
1.2.2. Canales ADC	3
1.3. Tipos de Datos	3
1.3.1. Estructura sensor_t	3
1.3.2. Enumeración alarm_state_t	4
1.4. Variables Globales	4
1.4.1. Sensores	4
1.4.2. Control de Sensores	4
1.4.3. LEDs	4
1.5. Funciones Privadas	5
1.5.1. ADC_ReadChannel()	5
1.5.2. Update_Sensors()	5
1.5.3. Update_Display()	5
1.5.4. Update_Alarm()	6
1.5.5. Process_Buttons()	6
1.6. Funciones Públicas	7
1.6.1. Monitor_Init()	7
1.6.2. Monitor_Loop()	7
2. Flujo de Operación	7

1. Módulo Monitor

1.1. Descripción General

El módulo `monitor.c` implementa un sistema de monitoreo de sensores para una placa STM32F411 con interfaz de LEDs, buzzer y botones. El sistema permite la lectura de dos sensores (LDR para luminosidad y NTC para temperatura), ajuste de niveles de alarma mediante potenciómetro, visualización en LEDs y activación de alarma sonora.

1.2. Constantes

1.2.1. Constantes NTC

```
1 #define R25      10000.0f // Resistencia a 25 deg C en ohmios
2 #define T25      298.15f // Temperatura de referencia en Kelvin
   (25 deg C)
3 #define BETA      3900.0f // Coeficiente Beta del termistor
```

Listing 1: Parámetros del termistor NTC

1.2.2. Canales ADC

```
1 #define CH_LDR  ADC_CHANNEL_0 // Canal para sensor LDR
2 #define CH_NTC  ADC_CHANNEL_1 // Canal para sensor NTC
3 #define CH_POT  ADC_CHANNEL_4 // Canal para potenciómetro
```

Listing 2: Asignación de canales ADC

1.3. Tipos de Datos

1.3.1. Estructura `sensor_t`

```
1 typedef struct {
2     float valor;           // Valor actual del sensor
3     float minimo;         // Valor mínimo de rango
4     float maximo;         // Valor máximo de rango
5     float nivel_alarma;    // Umbral de activación de
   alarma
6     int activated;         // Flag de activación
7     uint32_t time_activation; // Timestamp de activación
8     int value_flashing;    // Estado del parpadeo
9     uint32_t flashing_last_time; // Timestamp del último cambio
   de parpadeo
10 } sensor_t;
```

Listing 3: Estructura para almacenar datos de sensores

1.3.2. Enumeración alarm_state_t

```
1 typedef enum {  
2     ALARM_IDLE,           // Alarma inactiva  
3     ALARM_ACTIVE,        // Alarma activada/sonando  
4     ALARM_COOLDOWN       // Período de enfriamiento (10 segundos)  
5 } alarm_state_t;
```

Listing 4: Estados posibles de la alarma

1.4. Variables Globales

1.4.1. Sensores

```
1 sensor_t sensor_ldr = {0.0f, 0.0f, 100.0f, 50.0f, 0, 0, 0, 0};  
2 // valor=0, min=0%, max=100%, umbral=50%  
3  
4 sensor_t sensor_ntc = {0.0f, 25.0f, 30.0f, 27.5f, 0, 0, 0, 0};  
5 // valor=0, min=25 deg C, max=30 deg C, umbral=27.5 deg C
```

Listing 5: Instancias globales de sensores

1.4.2. Control de Sensores

```
1 uint8_t selected_sensor = 0;           // Sensor activo: 0=LDR, 1=  
    NTC  
2 uint32_t last_pot_value = 0xFFFF;     // Valor anterior del potenci  
    ómetro  
3 alarm_state_t alarm_state = ALARM_IDLE; // Estado de la  
    alarma  
4 uint32_t alarm_cooldown_start = 0;     // Timestamp del inicio del  
    cooldown  
5 uint8_t btn_izq_last = 1;              // Estado anterior botón  
    izquierdo  
6 uint8_t btn_der_last = 1;              // Estado anterior botón  
    derecho
```

Listing 6: Variables de control

1.4.3. LEDs

```
1 GPIO_TypeDef* LED_PORT[8] = {LED8_GPIO_Port, LED7_GPIO_Port,  
2                               LED6_GPIO_Port, LED5_GPIO_Port,  
3                               LED4_GPIO_Port, LED3_GPIO_Port,  
4                               LED2_GPIO_Port, LED1_GPIO_Port};  
5 uint16_t LED_PIN[8] = {LED8_Pin, LED7_Pin, LED6_Pin, LED5_Pin,  
6                        LED4_Pin, LED3_Pin, LED2_Pin, LED1_Pin};
```

Listing 7: Mapeo de pines LED

1.5. Funciones Privadas

1.5.1. ADC_ReadChannel()

```
1 static uint32_t ADC_ReadChannel(uint32_t channel)
```

Listing 8: Lectura de canal ADC

Descripción: Lee el valor digital de un canal ADC específico.

Parámetros:

- **channel:** identificador del canal ADC a leer

Retorna: Valor digital de 12 bits (0-4095)

Detalles:

- Configura el canal
- Inicia conversión ADC
- Espera a que la conversión termine
- Obtiene el valor y detiene el ADC

1.5.2. Update_Sensors()

```
1 static void Update_Sensors(void)
```

Listing 9: Actualización de valores de sensores

Descripción: Actualiza los valores de los sensores LDR, NTC y el potenciómetro.

Funcionalidad:

- **LDR:** Convierte lectura ADC a porcentaje (0-100 %), invertida (mayor ADC = menor luminosidad)
- **NTC:** Calcula temperatura en °C usando ecuación de Steinhart-Hart
- **Potenciómetro:** Ajusta el nivel de alarma del sensor actualmente seleccionado con histéresis de 30 unidades para evitar ruido

1.5.3. Update_Display()

```
1 static void Update_Display(void)
```

Listing 10: Actualización de visualización en LEDs

Descripción: Controla el encendido/apagado de LEDs según el valor del sensor activo.

Funcionalidad:

- Calcula cuántos LEDs deben estar encendidos basándose en el valor relativo del sensor
- Identifica el LED que corresponde al umbral de alarma
- Genera parpadeo a 10 Hz en el LED del umbral
- Actualiza el estado de todos los LEDs

1.5.4. Update_Alarm()

```
1 static void Update_Alarm(void)
```

Listing 11: Actualización del estado de alarma

Descripción: Gestiona la máquina de estados de la alarma.

Estados:

- **ALARM_IDLE:** Monitorea si algún sensor supera su umbral
- **ALARM_ACTIVE:** Alarma sonando, se mantiene hasta pulsar botón derecho
- **ALARM_COOLDOWN:** Período de 10 segundos antes de poder activar nuevamente

1.5.5. Process_Buttons()

```
1 static void Process_Buttons(void)
```

Listing 12: Procesamiento de entrada de botones

Descripción: Detecta pulsaciones de botones y ejecuta acciones correspondientes.

Acciones:

- **Botón Izquierdo:** Cambia el sensor activo entre LDR (0) y NTC (1)
- **Botón Derecho:** Desactiva la alarma y entra en período de cooldown si está activa

Implementación: Detecta flancos descendentes.

1.6. Funciones Públicas

1.6.1. Monitor_Init()

```
1 void Monitor_Init(void)
```

Listing 13: Inicialización del módulo

Descripción: Inicializa el módulo de monitoreo.

Funcionalidad:

- Apaga todos los LEDs (GPIO_PIN_RESET)
- Apaga el buzzer

Debe llamarse: Una vez en la función `main()` durante la inicialización del sistema.

1.6.2. Monitor_Loop()

```
1 void Monitor_Loop(void)
```

Listing 14: Bucle principal de monitoreo

Descripción: Función principal que debe llamarse periódicamente desde el bucle principal.

Ciclo de ejecución: Se ejecuta cada 20 ms (50 Hz)

Secuencia de operaciones:

1. `Process_Buttons()`: Procesa entrada de botones
2. `Update_Sensors()`: Actualiza valores de sensores
3. `Update_Alarm()`: Gestiona estado de alarma
4. `Update_Display()`: Actualiza visualización en LEDs

Temporizador: Utiliza `HAL_GetTick()` para asegurar una ejecución consistente cada 20 ms.

2. Flujo de Operación

El sistema opera de la siguiente manera:

1. **Lectura de Sensores:** El ADC lee continuamente los tres canales (LDR, NTC, Potenciómetro)

2. **Procesamiento:** Los valores se convierten a unidades significativas (porcentaje, °C)
3. **Control de Alarma:** Se comparan con el umbral ajustable mediante el potenciómetro
4. **Visualización:** Los LEDs muestran el nivel relativo, con parpadeo en el umbral
5. **Interacción:** Los botones permiten cambiar de sensor o silenciar la alarma